Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 24

Hash Tables

Motivation

- Suppose we are storing n distinct keys from a domain of size N
- Store them in an array A of size N
  - Store item with key i at A[i] (initially A[i] is marked "empty")
  - O(1) time for search/insert/delete!
- This is called Direct Addressing
- But $\Theta(N)$ space: realistic only when N is "small." Not feasible when the keys are say, integers ($N=2^{64}$ typically), or byte-strings of length 5 ($N=2^{40}$)
- Question: Can we match the speed of direct addressing using a reasonable amount of space?

Motivation

- Now, suppose we are storing n keys from a very large domain K
- Without direct addressing, we just need an array A of size m ≥ n to hold them
  - But searching would be slow, as a key could be sitting anywhere in the array
  - What if we could *calculate* its index as H(key) directly, using a "hash function" H: K → {0,...,m-1}?
  - When searching for key, we just compute H(key) and check if A[H(key)] has key stored there.
- Too good to be true! If |K| >> m, any function H: K → {0,..,m-1} will have to map many keys to the same index (collisions)
  - Hope: If we keep m slightly larger than n, then among the n items we get, there will only be a few collisions in each index. And, we can handle a few collisions.

Hash Tables

- Ingredients to make hash tables work
  - Good hash functions: how to ensure that there will only be a few collisions in each index (with high probability)
  - Handling increasing load: Grow the hash table as more keys inserted
  - Handling collisions: Two different paradigms
- Coming up today:
  - What are good hash functions

Hash Functions

- Central idea: If the function H is "random", and the load factor n/m is bounded by a constant (say < 1), then with high probability, there will be at most a few collisions in almost all indices
- What does with high probability mean?
  - There is a "universe of functions" $\mathcal{H}$ such that for any set of n keys chosen to be stored, *most* $H \in \mathcal{H}$ (say, more than 1-ε fraction) lead to low collisions
  - So if H is picked randomly from this universe, the probability that we pick a bad H (i.e., not with low collisions) is at most ε
    - However, for each H, there exist many key-sets with high collisions (in fact, if $|K| > mn$, there will be a key-set that is entirely mapped to a single index): No single H is universally good for all key-sets

Hash Functions

- What is a good universe of hash functions?
  - One that enforces "uniform hashing": each of the n keys is mapped to a uniformly random index, independent of the others
    - Then the expected number of items in any index is n/m, and further the probability that it is very far from expectation is very low
  - E.g., Universe of all functions from the domain of keys K to {0,...,m-1}
    - Picking a random H from this universe is exactly the same as uniform hashing (without limiting to n keys)
    - But no succinct description of a random function in this universe
  - We can "mimic" the universe of all functions using a smaller "pseudorandom" universe with efficiently representable functions

Hash Functions

- Pseudorandom: Has a similar effect as truly random.
  - Empirical (or Heuristic): Designed to work like random on a benchmark of inputs. No guarantees against adversarial inputs. Very fast.
  - Combinatorial: Provably as good as random against one-shot adversarial inputs. No guarantees against adaptive adversarial inputs.
  - Cryptographic: Strong guarantees. Only slightly slower, if relying on "lightweight-cryptography". Default in Python/Rust.
- Applications (e.g., those running on a web-server) which store untrusted inputs in hash tables are vulnerable to Denial of Service attacks, unless cryptographic hashes used

Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!

